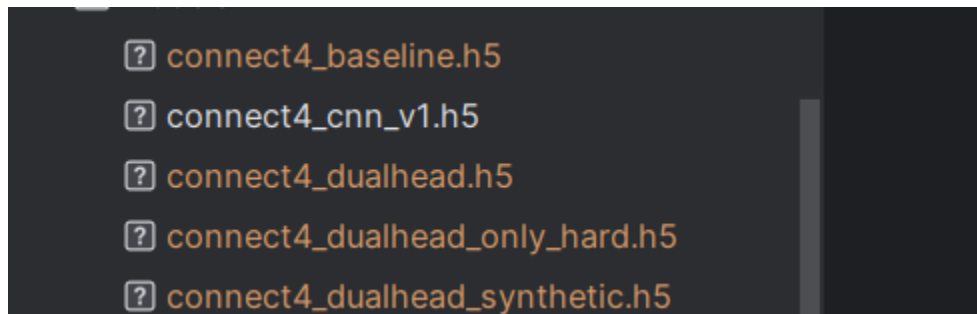


☐	Name	Time created	Last modified	Description	Tags
☐	Connect4_augmented_defensive_data	12/18/2025, 07:09:48 PM	12/18/2025, 07:09:48 PM	-	Add tags
☐	Connect4_defensive_data	12/18/2025, 06:40:02 PM	12/18/2025, 06:40:02 PM	-	
☐	Connect4_Behavioral_Cloning	12/18/2025, 05:49:28 PM	12/18/2025, 05:49:28 PM	-	
☐	Connect4_Strategy_Comparison	12/18/2025, 03:55:22 PM	12/18/2025, 03:55:22 PM	-	
☐	Connect4_Synthetic_Test	12/18/2025, 03:37:25 PM	12/18/2025, 03:37:25 PM	-	



First few experiments werent tracked in mlflow.

Extracting

The data is generated in the mongodb database. In the dataandai repository, in machine learning you can find `data_pipeline.py`. It generates data by playing through games from easy to hard against each other by running the `run_ai_games.py`. Then it runs [cleaning.py](#) which pulls json data from mongo db and flattens the data for the ml to be trained.

1. The mongo db stores the entire game state like this "'o a b ..." where o is empty a is player 1 and b is player2. so , we convert o to 0's, a to 1, and b to -1.
2. It extracts two columns from the mongodb as well, `best_move` (the best move decided by the mcts) and `win_label` (`game_result` puts 1 if this move was made by the winner of the match).

It then reaches the [preprocess.py](#) which normalizes the board, it loads the parquet file, removes empty boards, it transforms the raw data of player turns. Example, (knowing that 0 is empty, 1 is a and b is -1) if board is 0,1,-1 for player a's turn it stays the same. If it was player's b turn the board would flip to 0,-1,1 to make it easier for the cnn to understand during backpropagation when adjusting weights.

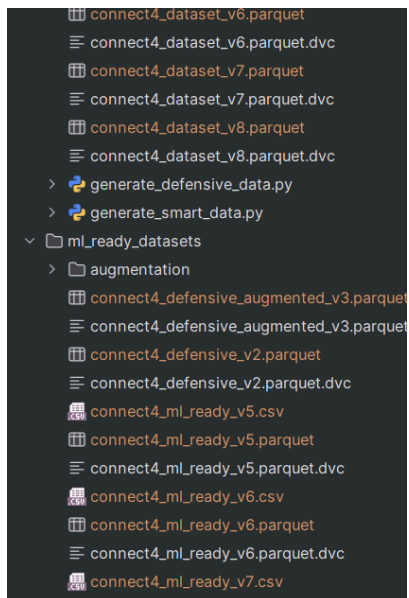
Cleaning

At first we trained on combinations of all different possibilities of difficulties. We learned that this was not a good idea as it tainted with the bias of the ml. It was learning from bad quality games like easy vs easy where moves are bad but they still won.

We implemented a `get_game_quality_weight` function in `cleaning_winners.py`, hard vs hard weight= 1 , hard vs medium weight = 0.5, other weight = 0.

Data Quality / Datasets / DVC

We used dvc to manage the lifecycle of our datasets and versioning, allowing us to track all versions in a shared space (dvc push would push to a shared google drive folder) . We had two different folders for datasets, `ml_ready`, where it went through all of the scripts and normal datasets where it didn't go through all of them. This was done to first analyze the data in the beginning to check quality and all the scripts were working correctly. But we stuck to it for the rest of the cycle.



Although this pipeline was complete and the model was successively trained, we were not entirely happy with the performance of the model. The model was not good at defending and playing long games, this is due to the fact that mcts games would end rather quickly which then the ml would be training on not so great data. The ml was also only reacting well to one half of the board, I believe this is because the mcts would go for the quickest route and would rarely play on the other side.

To mend this, we created synthetic data and augmented data to help aid the cnn in defending better. To augment the data, we mirrored the board . Connect4 is a horizontally symmetrical game, a winning strategy on the left half of the board would also work on the right side. `augment_Data.py` horizontally flips the board at every game state, the best move is also flipped to adjust for this .This successfully fixed the missing data that ensured that the cnn would perform in both halves of the board.

The script `generate_defensive_data.py` created random board states of scenarios where the opponent had 3 pieces in a row. It only saves boards where there is exactly one forced move to prevent a loss.

We also used a minmax algorithm to generate a few thousand games where minmax agent depth 4 played against a random opponent to get more quality for the training.

Training and performance

We tracked our training progress using ml flow, below you will find screenshots of the latest model. This model used a dual head architecture to give output on two tasks predicting the best move and predicting the winner. We used a 4,4 kernel size, three layers of 2d convolutions 128 -> 128 -> 64 filters with batch normalization and relu activation.

Layers 1 and 2 provide a wide search, the model can learn 128 types of different patterns, Layer 3 forces the model to summarize to most important information before making a final decision by removing unnecessary noise. The data is flattened into a feature vector and passed through fully connected layers that map into 7 column probabilities.

Batch normalization was used to stabilize variance between batches of only winning games or only defending games.

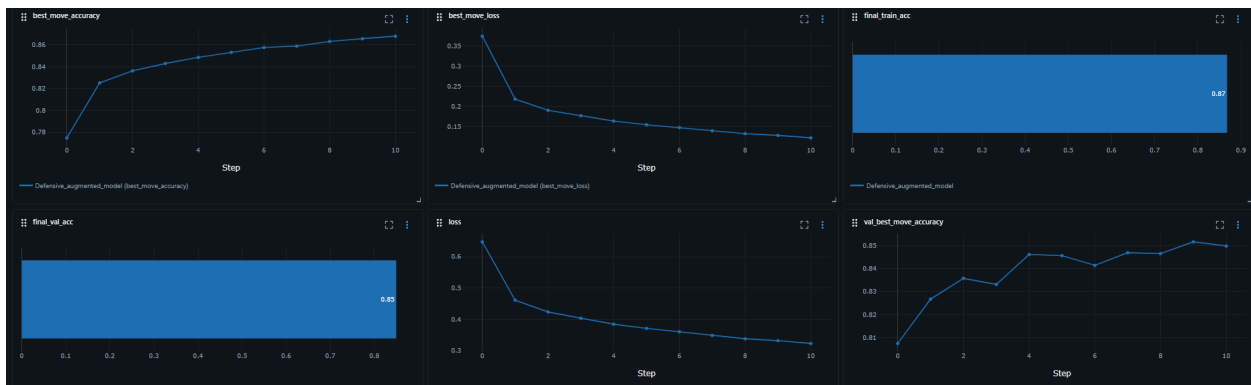
This was done to instead of looking at a flat list of 42 numbers (6x7 grid), it will look for 2d spatial patterns like pieces being next to each other, horizontally, vertically and diagonally. We used a 4,4 kernel size to make the cnn align with the game's win condition.

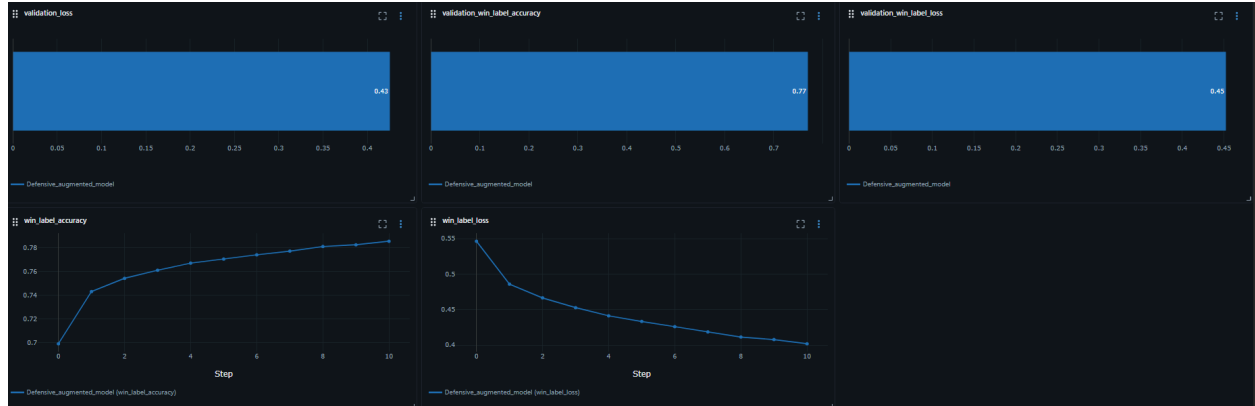
The model achieved high accuracy on the validation set, which confirms that the model can be used in unseen board states . Best move accuracy was 85%, Win Label accuracy was 76%.

Analysis of learning curves, Policy learning (best move) had a steep initial rise of 80% within the first few steps. The winner prediction is vastly more complex as its quite ambiguous to predict a winner on early and mid game board states. The `validation_loss` curve stabilises after 10 steps and it doesn't spike at the end which shows that its not overfitting, the mirroring provided enough variety to keep the model robust

The cnn model with an Adam optimizer (learning rate 0.001). The successful convergence of both heads simultaneously demonstrates that the shared Convolutional Neural Network (CNN) backbone successfully extracted features relevant to both attacking (Policy) and board evaluation (Value).

We used sigmoid activation function to get the output for win probability. Forcing the output to be a continuous value between 0 and 1. Even though our data was binary the model minimizes Binary cross Entropy loss by learning the average outcome of similar board states. An output of .75 implies that 75% of similar historical scenarios, the current player won. We applied weight learning from cleaning samples to this head .





Deployment & Experiments

To deploy our model into a playable environment, we used REST API. However, experimentation revealed that even high accuracy will still make mistakes, miss obvious wins and defenses. This is due to dataset quality, even though we did the best we could for augmentation, there were still limitations.

To solve this problem as we were not satisfied with the output, we implemented a hybrid approach, combining the cnn with a rule based system. The api has a safety net logic, which acts as a safety guard to not miss instant wins or blunders. Before finalizing any decisions, the system iterates through all legal columns using a fast rule based solver. If an immediate win is detected the system executes that move.

The system checks if the opponent has any winning response on the next turn. If the ai's chosen move allows an unblockable win for the opponent, the system overrides that decision and checks for ai's second best choice.

Future Improvements

We would have wanted to improve the win probability with a system that would look at future moves. Currently, the win probability endpoint returns the raw output from the model's value head. While useful, this is a static prediction based on a single board image. We would implement a look ahead system it would check for 1-2 possible moves in the future to check for possible losses and traps. We would also like to implement safety checks on the win probability percentage to more accurately depict scenarios like 100% chance to lose to be 100%.

We would also have wanted to generate more realistic data for ai in more scenarios to improve model performance.

VS MCTS

